

Quy nạp toán học và con đường giải bài

Zhang Kunwei

Trường Thực nghiệm Sơn Tây

Huấn luyện viên: Tang Wenbin, Hu Weidong

16 tháng 12 năm 2008

Nguồn gốc: Quy nạp toán học và con đường giải bài

IOI China candidate team papers / 2009 / Zhang Kunwei

Abstract

Bài viết tóm tắt lý thuyết liên quan đến quy nạp toán học, rồi kết hợp kinh nghiệm giải bài của tác giả để trình bày ứng dụng của quy nạp trong Olympic Tin học. Qua nhiều bài toán kinh điển và bài thi, bài viết bàn về chứng minh tính đúng đắn của thuật toán, thiết kế thuật toán xây dựng, quan hệ giữa quy nạp và tối ưu hóa thuật toán, cũng như phạm vi áp dụng và các mở rộng của lối tư duy quy nạp.

Contents

1	Dẫn nhập	2
2	Về quy nạp toán học	2
2.1	Một hồi tưởng ngắn	2
2.2	Các định lý, khái niệm và phương pháp cơ bản	2
2.3	Không chỉ tổng kết, mà còn khám phá	3
3	Ứng dụng trong chứng minh tính đúng đắn	4
3.1	Thuật toán tham lam	4
3.2	Các thuật toán khác	4
4	Ứng dụng trong thuật toán xây dựng	5
4.1	Khôi phục cấu trúc dữ liệu	5
4.2	Xây dựng chiến lược và phương án	6
5	Quy nạp toán học và tối ưu hóa thuật toán	6
5.1	Chọn khéo đối tượng quy nạp	6
5.2	Hoàn thiện cơ sở quy nạp	7
5.3	Chọn hướng quy nạp cẩn thận	7
5.4	Tăng cường giả thiết quy nạp	8

6 Tác dụng gợi mở và giá trị thẩm mỹ	8
7 Vấn đề và hạn chế	9
7.1 Về mặt lý thuyết	9
7.2 Về mặt ứng dụng	9
7.3 Các lớp bài không thuận lợi	9
8 Lời cuối	9
9 Nguồn bài tập	9

1 Dẫn nhập

Đạo sinh một, một sinh hai, hai sinh ba, ba sinh vạn vật.

Lão Tử, *Đạo Đức Kinh*

Trong khi học thuật toán, ta thường tự hỏi: những thuật toán tự nhiên và tinh tế ấy được nghĩ ra như thế nào, và vì sao chúng đúng? Đằng sau mỗi lời giải không chỉ có kỹ thuật cụ thể, mà còn có một cách nhìn. Nếu muốn tìm một điểm khởi đầu cho “đạo” giải bài, quy nạp toán học là một lựa chọn rất đáng tin.

Quy nạp không chỉ dùng để viết phần chứng minh sau khi đã biết đáp án. Trong quá trình đoán, thử, tách bài toán và xây dựng thuật toán, ta cũng nhiều lần dùng tư tưởng: giải được trường hợp nhỏ, rồi tìm cách mở rộng có kiểm soát sang trường hợp lớn hơn.

2 Về quy nạp toán học

2.1 Một hồi tưởng ngắn

Nguyên lý quy nạp toán học có thể truy ngược tới các công trình số học thời Phục hưng, chẳng hạn tác phẩm *Arithmeticon libri duo* của Francesco Maurolico năm 1494, và sau này được đặt trên nền hệ tiên đề Peano. Từ khi ra đời, nó đã được chú ý rộng rãi vì là một cách xử lý chung, ngắn gọn cho các mệnh đề liên quan tới số nguyên dương. Ngay cả với những bài toán khó, dùng quy nạp thường giúp đơn giản hóa mô tả, tách rõ bản chất, rồi mở đường cho lời giải tiếp theo.

Do bản chất đệ quy, tính xây dựng và cả giá trị thẩm mỹ của nó, quy nạp toán học thường xuất hiện trong sách phổ thông và trong các kỳ thi học sinh giỏi. Trong Tin học, nơi các bài toán số học, logic, topo, lý thuyết số và tổ hợp liên tục xuất hiện, ta càng có lý do tin rằng quy nạp toán học, cũng như các biến thể như quy nạp cấu trúc, có đất dụng võ trong thiết kế thuật toán.

2.2 Các định lý, khái niệm và phương pháp cơ bản

Quy nạp toán học thứ nhất. Với một mệnh đề $P(n)$ trên các số tự nhiên dương, nếu

$$P(1) = \text{True}, \quad P(n) \Rightarrow P(n + 1) \quad (n \in \mathbb{N}^+),$$

thì $P(n)$ đúng với mọi n . Có thể hình dung các số tự nhiên là một dãy quân domino: điều kiện $P(1) = \text{True}$ là đẩy quân đầu tiên, còn bước $P(n) \Rightarrow P(n + 1)$ bảo đảm cứ quân trước ngã thì quân sau cũng ngã. Nhờ đó cả dãy domino lần lượt đổ xuống.

Quy nạp toán học thứ hai. Nếu với mọi n ,

$$(\forall k < n, P(k)) \Rightarrow P(n),$$

thì $P(n)$ đúng với mọi n . Dạng này linh hoạt hơn vì ở bước n ta có thể dùng mọi kết quả đã chứng minh trước đó, không chỉ trường hợp $n - 1$. Nó cho phép giảm kích thước bài toán đi một nửa, chia thành nhiều phần nhỏ, hoặc chọn một trường hợp con thuận lợi. Trên cấu trúc tuyến tính, ngoài một số bài rất đơn giản dùng trực tiếp dạng thứ nhất, phần lớn các chứng minh và thiết kế thuật toán thực tế đều gần với dạng thứ hai hơn.

Quy nạp cấu trúc. Quy nạp cấu trúc là biến thể thường gặp trong logic, khoa học máy tính và lý thuyết đồ thị. Ta không nhất thiết quy nạp theo một số tuyến tính, mà theo một thứ tự topo của cấu trúc, thường là kích thước bài toán. Với cây, ta có thể tách các cây con; với đồ thị, ta có thể bỏ một cạnh hoặc một đỉnh; với biểu thức, ta xét các biểu thức con. Tính đúng đắn dựa trên nguyên lý phản ví dụ nhỏ nhất: nếu có phản ví dụ thì tồn tại phản ví dụ nhỏ nhất, nhưng lập luận quy nạp lại tạo ra một phản ví dụ nhỏ hơn, mâu thuẫn.

Trong quy nạp cấu trúc, cách định nghĩa “kích thước” đặc biệt quan trọng. Nó cũng rất linh hoạt. Nếu ta không muốn bước quy nạp phải chống lại quá nhiều khó khăn cùng lúc, cần mạnh dạn tăng cường giả thiết quy nạp, tách các đại lượng bề ngoài giống nhau thành những thông tin trạng thái riêng biệt. Nhiều ví dụ phía sau sẽ dùng đúng thủ pháp này.

Ví dụ 1: mã Prüfer cho cây gán nhãn. Công thức Cayley nói rằng có n^{n-2} cây không gốc khác nhau trên n đỉnh gán nhãn. Để chứng minh công thức này, Heinz Prüfer năm 1918 đưa ra mã Prüfer nổi tiếng. Về cốt lõi, mã này khẳng định tồn tại một song ánh giữa cây không gốc trên n đỉnh với nhãn lấy từ m giá trị khả dĩ và dãy độ dài $n - 2$, mỗi phần tử nằm trong $1, \dots, m$.

Lập luận quy nạp rất tự nhiên: theo n , chọn lá có nhãn nhỏ nhất, ghi nhãn đỉnh kề với nó vào đầu dãy, xóa lá đó, rồi dùng giả thiết quy nạp cho cây còn lại. Công thức Cayley chỉ là trường hợp đặc biệt khi $m = n$. Nếu cố quy nạp trực tiếp trên công thức đếm, số n xuất hiện quá nhiều chỗ và làm chứng minh nặng hơn hẳn; quy nạp cấu trúc cho ta đúng đối tượng cần thao tác.

2.3 Không chỉ tổng kết, mà còn khám phá

Trong giải bài, một phần là đoán và khám phá, phần còn lại là tổng kết và chứng minh. Quy nạp thường bị xem như công cụ của giai đoạn thứ hai, nhưng thực ra khi ta tìm cách loại bỏ một phần tử, thu nhỏ dữ liệu, hoặc tăng dần kích thước nghiệm, ta đã dùng tư duy quy nạp.

Vì vậy mục đích của bài viết này không chỉ là nhắc lại một công cụ chứng minh. Tác giả muốn gợi lại nhiệt tình của người đọc với mặt “khám phá” của quy nạp: nhiều khi trong lúc chưa biết lời giải, chính câu hỏi “nếu bỏ bớt một phần thì sao” hoặc “nếu đã giải được phần trước thì thêm phần này như thế nào” sẽ dẫn ta tới thuật toán.

Ví dụ 2: bài toán người nổi tiếng. Cho đồ thị có hướng dưới dạng ma trận kề, cần tìm đỉnh chỉ có cạnh đi vào và không có cạnh đi ra, với số câu hỏi ít nhất. Xác định trực tiếp ứng viên có thể tồn cả ma trận, nhưng loại một đỉnh không thể là đáp án thì dễ hơn. Hỏi một cạnh giữa hai ứng viên: nếu cạnh tồn tại, loại đỉnh xuất phát; nếu không, loại đỉnh kết thúc. Mỗi câu hỏi giảm kích thước bài toán đi 1, rồi giả thiết quy nạp tiếp tục xử lý tập còn lại. Thuật toán tương tác chuẩn này chính là một quá trình quy nạp.

3 Ứng dụng trong chứng minh tính đúng đắn

Trong luyện tập ta thường lấy AC làm điểm kết thúc, nhưng hiểu vì sao thuật toán đúng mới là nền tảng vững chắc. Đặc biệt với tham lam, chứng minh thường quyết định thuật toán có đáng tin hay không. Quy nạp là một trong các công cụ chứng minh phổ biến nhất của thuật toán.

3.1 Thuật toán tham lam

Ví dụ 3: Robbers. N tên cướp chia M đồng vàng theo tỉ lệ $X_i / \sum X_i$, nhưng vàng không thể chia nhỏ. Gọi K_i là số vàng tên thứ i nhận được; cần làm nhỏ sai lệch tổng thể. Một chiến lược tự nhiên là trước hết chia phần nguyên theo tỉ lệ, rồi mỗi lần cấp một đồng vàng cho người có độ thiếu hụt

$$\Delta_i = \frac{X_i}{\sum X_i} - \frac{K_i}{\sum K_i}$$

lớn nhất.

Có thể chứng minh bằng quy nạp rằng sau khi đã phát m đồng vàng,

$$\max_i \Delta_i - \min_i \Delta_i \leq \frac{1}{M}.$$

Ở cơ sở, việc chia tỉ lệ ban đầu thỏa bất đẳng thức. Khi thêm một đồng cho người đang thiếu nhất, nếu người đó chưa vượt lên thành người “dư” nhất thì khoảng cách chỉ giảm. Nếu có vượt, giá trị mới cũng chỉ tăng thêm đúng phần đóng góp của một đồng vàng, nên bất đẳng thức vẫn giữ được. Phần còn lại là chứng minh mọi nghiệm tối ưu cũng phải có tính cân bằng tương tự, từ đó thuật toán tham lam là đúng.

Cụ thể hơn, giả sử người nhận thêm một đồng trở thành người có Δ_i lớn nhất. Khi đó có thể viết

$$\max_i \Delta'_i = \min_i \Delta_i + \frac{1}{\sum K_i} = \min_i \Delta_i + \frac{1}{M} \leq \min_i \Delta'_i + \frac{1}{M},$$

nên kết luận quy nạp vẫn giữ. Để nối kết bất biến này với tối ưu, có thể dùng phương pháp điều chỉnh: trước hết chứng minh nghiệm tối ưu cũng phải thỏa điều kiện cân bằng trên, sau đó chứng minh nếu chỉ xét các hoán vị khác nhau của Δ_i thì nghiệm tối ưu tồn tại và duy nhất. Chi tiết này không liên quan trực tiếp tới quy nạp nên bản gốc chỉ nêu gợi ý.

3.2 Các thuật toán khác

Ví dụ 4: Think Positive. Một hành tinh có n ngày trong năm; ngày vui có giá trị $+1$, ngày buồn có giá trị -1 . Chọn ngày đầu năm sao cho mọi tổng tiền tố trong năm đều dương. Bài toán trở thành đếm số vị trí bắt đầu trong một dãy vòng sao cho tổng tiền tố luôn dương.

Một cách nghĩ ban đầu là thử lần lượt mọi điểm bắt đầu và dùng hàng đợi đơn điệu để duy trì giá trị nhỏ nhất của dãy tổng tiền tố trên vòng tròn. Cách này cũng tuyến tính, nhưng cài đặt phức tạp hơn và dễ sai. Quy nạp mạnh cho một kết luận gọn hơn nhiều: với mọi dãy ngắn hơn, số điểm bắt đầu hợp lệ chỉ phụ thuộc vào tổng của các phần tử; nếu tổng âm thì đáp án là 0, còn khi tổng dương thì số điểm hợp lệ được suy ra theo quy tắc giảm kích thước.

Xét quy nạp mạnh theo độ dài dãy. Nếu tồn tại cặp liên tiếp $+1, -1$, xóa cặp này không đổi tổng toàn dãy. Hai vị trí vừa xóa không thể là điểm bắt đầu hợp lệ, còn các vị trí khác giữ nguyên điều kiện tiền tố hoặc trở nên tốt hơn; vì vậy bài toán giảm đi 2. Nếu không có cặp $+1, -1$, hoặc dãy toàn -1 nên đáp án là 0, hoặc dãy toàn $+1$ nên mọi vị trí đều hợp lệ. Lập luận này cho một thuật toán tuyến tính rất đơn giản.

4 Ứng dụng trong thuật toán xây dựng

Thuật toán xây dựng thường khó vì không có khuôn mẫu cố định. Quy nạp giúp ta tìm điểm bám: xây được nghiệm nhỏ, rồi thêm một phần tử hoặc nối thêm một khối sao cho bất biến vẫn đúng.

Khác với nhiều phương pháp có hình thức tương đối cố định, thuật toán xây dựng thường đòi hỏi nhiều sáng tạo hơn. Cũng vì vậy, nếu biết chắc cần xây dựng mà vẫn không biết bắt đầu từ đâu, tư tưởng quy nạp có thể đóng vai trò “đường lùi”: tìm một cấu trúc con đã giải được, rồi tìm thông tin bổ sung đủ mạnh để nối cấu trúc mới vào mà không phá điều kiện bài toán.

4.1 Khôi phục cấu trúc dữ liệu

Ví dụ 5: Set Cover từ bài toán phủ đoạn. Bài toán phủ đoạn chọn ít điểm nguyên nhất sao cho mỗi đoạn cho trước chứa ít nhất một điểm. Nếu một instance của set cover được tạo ra từ bài toán phủ đoạn, ta muốn khôi phục đủ cấu trúc tuyến tính để dùng tham lam.

Với các đoạn không chứa nhau, duyệt từ trái sang phải và khi gặp đoạn chưa được phủ thì chọn điểm phải nhất còn phủ được nó. Vì set cover tổng quát là NP-đầy đủ, trọng tâm là khôi phục quan hệ tuyến tính ẩn. Ta chỉ cần biết thứ tự tương đối của các đoạn và đầu phải của mỗi đoạn. Giả thiết quy nạp có thể là: trong một thành phần liên thông theo quan hệ giao nhau, ta đã biết vị trí các đoạn đã xử lý và biết một đoạn A giao với đoạn mới X . Khoảng cách giữa A và X suy ra từ sai khác đối xứng của hai tập. Để biết X nằm phía nào của A , dùng thêm một đoạn đã biết hướng B ; X và B ở cùng phía của A khi phần giao của chúng với A có quan hệ bao hàm. Bằng cách tăng cường giả thiết quy nạp, ta lần lượt dựng lại cấu trúc cần thiết.

Ở đây ta chia bài toán thành từng thành phần liên thông theo quan hệ giao nhau, vì hai nhóm đoạn đôi một không giao nhau có thể đặt cách nhau tùy ý mà không ảnh hưởng lời giải. Đoạn đầu tiên trong mỗi thành phần có thể đặt tại vị trí 0. Nếu không còn chọn được X giao với một đoạn đã biết, thành phần hiện tại đã kết thúc. Khi đã dựng được vị trí, đầu phải của một đoạn A có thể lấy bằng giao của các tập tương ứng với A và mọi đoạn nằm bên phải, giao với A ; một phần tử bất kỳ trong giao đó đóng vai trò điểm phải nhất cần cho thuật toán tham lam. Toàn bộ quá trình cho thấy giả thiết quy nạp linh hoạt là phần quan trọng nhất của lời giải.

Ví dụ 6: Roman Roads. Cho quan hệ kề giữa các con đường, cần xác định liệu chúng có thể là các cạnh của một cây đường La Mã hay không và dựng cây đó. Với cây, quan hệ giữa cạnh và đỉnh có tính đệ quy rõ rệt. Gọi X là cạnh nối đỉnh x với cha p , và P là cạnh nối p với cha của nó. Giả thiết quy nạp: khi biết x , X và P , ta có thể dựng cây con gốc x .

Các cạnh kề với X chia làm hai loại: cạnh cùng nối tại p , và cạnh nối tại x . Các cạnh loại đầu đã thuộc phần phía trên; các cạnh loại sau tạo ra các cây con cần tiếp tục quy nạp. Cạnh P giúp phân biệt hai loại: cạnh nối tại p kề với P , còn cạnh nối tại x thì không. Từ đó có thể dùng phép hiệu tập để tách nhánh và kiểm tra tính hợp lệ.

Giả thiết này ngầm chọn hướng xây dựng từ trên xuống. Về mặt logic, cũng có thể khôi phục từ dưới lên bằng cách liên tục xóa lá; người đọc có thể tự thử cách đó. Tuy nhiên dữ liệu đầu vào không cho ta thông tin trực tiếp về lá, nên hướng dưới lên phải tìm lá lặp đi lặp lại và còn phải kiểm tra tính đúng sai sau khi dựng xong. Hướng từ trên xuống cho phép vừa dựng vừa kiểm tra, còn điều kiện cần và đủ có thể phát biểu gọn: mọi nhánh đều có và chỉ có một gốc.

4.2 Xây dựng chiến lược và phương án

Trong nhiều bài xây dựng, ta không dựng lại một cấu trúc đã sẵn, mà phải đóng vai người tư vấn: đưa ra một chiến lược, một kế hoạch hoặc một dãy thao tác. Khi đó điều quan trọng là tìm một tính chất vừa đúng cho toàn cục, vừa có thể duy trì trên từng phần nhỏ. Tính chất ấy sẽ trở thành chiếc cầu nối giữa giả thiết quy nạp và kết luận quy nạp.

Ví dụ 7: Royal Federation. Trong một cây thành phố, cần chia thành các bang, mỗi bang có từ B đến $3B$ thành phố và có một trung tâm hành chính sao cho mọi thành phố trong bang đi tới trung tâm mà không ra khỏi bang. Hãy xem cây có gốc. Giả thiết quy nạp: với một cây con nhỏ hơn, ta chia được các phần có kích thước $B \leq x \leq 2B$, còn lại một tập thành phố nối với gốc và có kích thước $x \leq B$.

Áp dụng giả thiết cho từng cây con của gốc, rồi gom các phần dư. Mỗi khi đủ B thành phố thì tạo một bang với trung tâm là gốc hiện tại. Vì mỗi phần dư con có kích thước không quá B , bang mới không vượt $2B$; phần dư cuối cùng không quá B . Ở bước cuối cùng, gộp phần dư vào bang kế nó để đạt giới hạn $3B$. Điểm mấu chốt là chọn đúng bất biến “phần còn lại nối với gốc và có kích thước không quá B ”.

Ví dụ 8: Electricity. Mạng điện là một DAG. Chính phủ muốn đảo hướng một số cạnh, mỗi ngày một cạnh, và tại mọi thời điểm đồ thị vẫn phải không có chu trình. Giả sử không có cạnh song song. Quy nạp theo số đỉnh: chọn một đỉnh A không có cạnh ra trong trạng thái cuối. Các cạnh đi vào A không cần đổi, còn các cạnh đi ra từ A cần đảo.

Để đảo các cạnh xuất phát từ A , dùng một quy nạp phụ theo số cạnh còn cần đổi. Chọn cạnh $A \rightarrow B$ có đỉnh cuối B đứng sớm nhất trong thứ tự topo của đồ thị ban đầu, rồi đảo thành $B \rightarrow A$. Nếu tạo chu trình, chu trình phải có dạng

$$A \rightarrow C \rightarrow \dots \rightarrow B \rightarrow A.$$

Nhưng khi đó C phải đứng sau B trong thứ tự topo, mâu thuẫn với đường đi $C \rightarrow \dots \rightarrow B$. Do đó bước đảo này an toàn, và phần còn lại giao cho giả thiết quy nạp. Từ phân tích này suy ra thuật toán gọn: sắp các cạnh cần đổi theo thứ tự nghịch của đỉnh đầu trong trạng thái cuối, rồi theo thứ tự thuận của đỉnh cuối trong trạng thái đầu.

Phân tích thêm cho thấy không cần topo-sort nhiều lần. Khi xử lý xong A , các cạnh giữa những đỉnh còn lại không đổi, nên thứ tự topo của phần còn lại trong cả trạng thái đầu và trạng thái cuối vẫn giữ nguyên. Vì thế một lần sắp thứ tự là đủ. Đây là một ví dụ điển hình cho việc bước quy nạp chính lại cần một quy nạp phụ bên trong.

5 Quy nạp toán học và tối ưu hóa thuật toán

Tối ưu hóa ở đây không phải tối ưu hằng số bằng hợp ngữ, mà là chọn cách cài đặt, mô hình và hướng xử lý hiệu quả hơn. Nhiều cải tiến thuật toán có thể được nhìn dưới lăng kính quy nạp: chọn lại đối tượng quy nạp, cơ sở quy nạp, hướng quy nạp, hoặc tăng cường giả thiết.

5.1 Chọn khéo đối tượng quy nạp

Ví dụ 9: Explaining the Stock Market. Cho một dãy giá, cần chia thành ít dãy con đơn điệu tăng hoặc giảm nhất, với điều kiện một ngày đơn lẻ không được tính là dãy đơn điệu trừ các trường hợp đặc biệt. Nếu quy nạp theo độ dài còn lại bằng cách mỗi lần lấy ra một dãy con đơn điệu, ta có một tìm kiếm đúng nhưng nhiều trạng thái.

Thực ra điều kiện “một ngày đơn lẻ không tính là dây đơn điệu” chủ yếu xử lý những trường hợp quá ít ngày, vì ta có thể lấy một ngày khỏi một dây con bất kỳ để ghép vào một ngày đơn lẻ nếu cần. Bởi vậy nó nên được xem là trường hợp đặc biệt. Thuật toán tìm kiếm trực tiếp tương ứng với giả thiết quy nạp: “đã biết cách tối ưu cho một dây ngắn hơn”. Mỗi bước thử rút ra một dây con đơn điệu rồi giải phần còn lại. Cách này đúng và có thể đủ nhanh trong giới hạn gốc, nhưng mỗi chuyển trạng thái phải liệt kê nhiều dây con đơn điệu, nên khung quy nạp này không phải điểm tối ưu tốt.

Trước hết xét biến thể chỉ cho dây tăng. Duyệt từ trái sang phải; khi thêm một giá mới, ta đặt nó vào dây tăng có phần tử cuối lớn nhất nhưng vẫn nhỏ hơn giá mới. Dây có đuôi nhỏ hơn còn nhiều tiềm năng hơn, nên không nên dùng nó nếu không cần. Với cả dây tăng và giảm, nghiệm cục bộ gồm các đuôi của các dây hiện có; mỗi phần tử mới thử ghép vào một dây tăng hoặc một dây giảm, rồi giữ nghiệm có số dây ít hơn. Như vậy đối tượng quy nạp là tiền tố đã xử lý và “nghiệm còn nhiều tiềm năng nhất”, thay vì toàn bộ phần dây còn lại.

Nói chính quy hơn, nếu tại một thời điểm hai nghiệm bộ phận có các đuôi dây $\{a_i\}$ và $\{b_i\}$, thỏa $a_i \leq b_i$ với mọi i , thì bất kỳ cách mở rộng tối ưu nào của $\{b_i\}$ cũng có thể mô phỏng trên $\{a_i\}$, thậm chí có thể dùng ít dây hơn nếu một dây của $\{b_i\}$ không được mở rộng nữa và $a_i = -\infty < b_i$. Vì vậy chỉ cần giữ nghiệm “có nhiều tiềm năng nhất”. Với cả tăng và giảm, không gian tìm kiếm lý thuyết vẫn có thể là $\mathcal{O}(2^n)$, nhưng sau khi thêm cắt tỉa tối ưu, mọi dữ liệu trong bài gốc đều chạy tức thì.

5.2 Hoàn thiện cơ sở quy nạp

Ví dụ 10: tối ưu quicksort. Quicksort có thể hiểu là thuật toán quy nạp: phân hoạch dãy rồi sắp xếp hai phần nhỏ hơn. Nhưng với dãy rất ngắn, chi phí đệ quy và phân hoạch có thể lớn hơn lợi ích. Ta mở rộng cơ sở quy nạp: nếu độ dài nhỏ hơn một ngưỡng, dùng insertion sort hoặc heap sort. Cơ sở quy nạp xuất hiện rất nhiều lần trong quá trình chạy, nên đầu tư tối ưu ở đây thường có hiệu quả. Introsort và nhiều biến thể sắp xếp thực tế dùng đúng tư tưởng này.

Điểm đáng chú ý là tối ưu hóa cơ sở quy nạp khác với sửa vài hằng số ở phần thân thuật toán. Cơ sở được gọi rất nhiều lần, nên chỉ cần làm nó “hoàn thiện” hơn một chút cũng có thể giảm số lần đệ quy và cải thiện tốc độ tổng thể. Các kỹ thuật tiền xử lý để tăng tốc cũng thường có cùng tinh thần: chấp nhận một chi phí nhỏ, có tổ chức, ở phần nền để các bước lặp lại phía sau nhẹ hơn.

5.3 Chọn hướng quy nạp cẩn thận

Ví dụ 11: quy tắc Horner. Tính đa thức $\sum_{i=0}^N a_i x^i$. Nếu quy nạp theo tổng các hạng từ bậc thấp lên, ta còn phải duy trì x^k . Đổi hướng từ bậc cao xuống cho

$$\sum_{i=0}^N a_i x^i = (\cdots ((a_N x + a_{N-1})x + a_{N-2}) \cdots)x + a_0,$$

ta được quy tắc Horner, giảm số phép nhân và trạng thái phụ.

Lý do nằm ở hướng quy nạp: trong thuật toán quy nạp, phần xử lý trước thường phải chịu nhiều thao tác hơn. Với đa thức, các hạng bậc cao vốn cần nhiều phép nhân hơn nếu làm trực tiếp, nên đưa chúng vào trước sẽ làm phép nhân được dùng lại tự nhiên.

Ví dụ 12: xây dựng heap nhị phân. Từ một mảng vô thứ tự, xây heap từ trên xuống có độ phức tạp $\Theta(n \log n)$, còn xây từ dưới lên là $\Theta(n)$. Nhìn theo quy nạp, phần xử lý sớm thường xảy ra khi kích thước bài toán nhỏ, nên có lợi hơn. Heap có nhiều nút ở đáy; chọn hướng từ dưới lên đặt lợi thế này vào đúng phần chiếm số lượng lớn.

Nếu chỉ nhìn vào hai phép phân tích độ phức tạp, ta dễ thấy kết quả nhưng khó thấy bản chất. Từ góc nhìn quy nạp, các bước đầu luôn làm việc trên bài toán nhỏ hơn và thường nhanh hơn; heap lại “nặng đáy, nhẹ đỉnh”. Hướng từ dưới lên đem lợi thế của các bước nhỏ đến đúng nơi có nhiều nút nhất.

5.4 Tăng cường giả thiết quy nạp

Ví dụ 13: Yet Another Digit. Giữ trọng số nhị phân nhưng cho phép mỗi chữ số là 0, 1, 2; số cách biểu diễn một số n theo dạng này gọi là $f(n)$. Công thức trực tiếp là

$$f(n) = \begin{cases} f(n/2) + f(n/2 - 1), & n \equiv 0 \pmod{2}, \\ f(\lfloor n/2 \rfloor), & n \equiv 1 \pmod{2}. \end{cases}$$

Với n rất lớn, cách này vẫn khó vì các bài toán con chồng lấn và không thuận tiện cho số nguyên lớn. Quan sát $f(n)$ thường đi cặp với $f(n-1)$, đặt $g(n) = f(n-1)$ và tăng cường giả thiết quy nạp thành: đã biết

$$f(\lfloor n/2 \rfloor), \quad g(\lfloor n/2 \rfloor).$$

Khi đó

$$f(n) = \begin{cases} f(\lfloor n/2 \rfloor) + g(\lfloor n/2 \rfloor), & n \equiv 0 \pmod{2}, \\ f(\lfloor n/2 \rfloor), & n \equiv 1 \pmod{2}, \end{cases}$$

và

$$g(n) = \begin{cases} g(\lfloor n/2 \rfloor), & n \equiv 0 \pmod{2}, \\ g(\lfloor n/2 \rfloor) + f(\lfloor n/2 \rfloor), & n \equiv 1 \pmod{2}. \end{cases}$$

Cài đặt chỉ cần hai biến f, g và quét biểu diễn nhị phân của số lớn từ cao đến thấp: gặp bit 0 thì cập nhật $f := f + g$, gặp bit 1 thì cập nhật $g := f + g$. Độ phức tạp trở thành $\Theta(\log n)$.

Nếu dùng công thức trực tiếp và phân tích thô, thời gian vẫn quá lớn cho các số có thể dài tới 10^{100} . Ngay cả khi nhớ hóa để xử lý bài toán con chồng lấn, không gian cũng không chấp nhận được. Mấu chốt là “cục bộ hóa bộ nhớ”: nhìn ra cách các bài toán con xuất hiện theo cặp, rồi đưa chính cặp đó vào giả thiết quy nạp. Khi nhìn theo biểu diễn nhị phân, giả thiết mới gần giống quy nạp bậc nhất trên từng tiền tố của số.

6 Tác dụng gợi mở và giá trị thẩm mỹ

Các ví dụ trên cố gắng bóc tách những “nét bút thần” trong lời giải thành những bước tự nhiên hơn: chọn cấu trúc nhỏ, xác định bất biến, mở rộng có kiểm soát. Thuật toán đẹp không nhất thiết là thuật toán gây bất ngờ, mà thường là thuật toán đơn giản, đi thẳng vào bản chất. Quy nạp là một trong những cách giúp ta tìm sự đơn giản ấy.

Không thể tồn tại một “thuật toán tìm mọi thuật toán” theo nghĩa tuyệt đối, nhưng ta vẫn có thể rèn luyện các chiến lược tư duy chung. Nếu các định lý và kỹ thuật cụ thể là con đường trong từng bài, thì quy nạp là một phương pháp định hướng trên con đường đó.

Tác giả nhắc tới việc Viện sĩ Wu Wenjun đưa chứng minh máy vào hình học: nếu có “phương pháp chứng minh”, hẳn nhiều người sẽ tự hỏi liệu có thể có “thuật toán của thuật toán” để tìm lời giải hay không. Ở nghĩa tuyệt đối, điều đó không thể thực hiện; nhưng ở nghĩa thực hành, việc rèn các chiến lược tư duy phổ quát vẫn rất có giá trị. Quy nạp chỉ là một phần nổi của tảng băng trong các chiến lược ấy.

7 Vấn đề và hạn chế

7.1 Về mặt lý thuyết

Tính đúng đắn của quy nạp dựa vào tiên đề quy nạp trong hệ Peano. Theo định lý bất toàn của Gödel, một hệ tiên đề đủ mạnh không thể tự chứng minh toàn bộ tính đúng đắn của chính nó trong nội bộ hệ. Trong phạm vi thuật toán thi đấu, ta hầu như không gặp những vấn đề nền tảng kiểu này, nên quy nạp vẫn là công cụ suy luận đáng tin. Dù vậy, đặc điểm “từ hữu hạn đi tới vô hạn” mà không có chứng minh trực tiếp bên ngoài hệ tiên đề vẫn là một giới hạn triết học cần biết.¹

7.2 Về mặt ứng dụng

Nếu gặp mọi bài đều viết đầy đủ giả thiết quy nạp, cơ sở và bước chuyển, ta có thể tự tạo thêm chi phí không cần thiết. Mục tiêu là dùng linh hoạt: khi cần chứng minh tham lam, dựng thuật toán, hoặc kiểm soát một bất biến khó, quy nạp phải xuất hiện đúng lúc; khi mô hình đã rõ, ta không cần hình thức hóa quá mức.

Nói cách khác, từ “dùng được” và “biết dùng” phải tiến tới “dùng linh hoạt” và “dùng khéo”. Trong thi đấu, hình thức trình bày chỉ nên phục vụ tư duy và kiểm chứng, không nên trở thành gánh nặng làm chậm chính quá trình giải bài.

7.3 Các lớp bài không thuận lợi

Một số lớp bài không dễ tiếp cận bằng quy nạp, chẳng hạn quy hoạch tuyến tính, quy hoạch nguyên và các bài quy về chứng như một số mô hình luồng mạng; một số bài tối ưu hóa toàn cục; phần lớn thuật toán xấp xỉ hoặc không hoàn hảo; thuật toán số; và các bài dựa chủ yếu trên ma trận hoặc mô hình toán học khác. Danh sách này không tuyệt đối, vì kinh nghiệm của tác giả có hạn, nhưng nó nhắc ta rằng quy nạp là một công cụ mạnh chứ không phải chìa khóa vạn năng.

8 Lời cuối

Tác giả từng đọc sách nghiên cứu giải bài của thầy Shan Zun, trong đó nhấn mạnh rằng lời giải nên tự nhiên, đơn giản và đi vào lõi vấn đề. Bài viết ban đầu chỉ định trình bày một chuyên đề về quy nạp toán học, nhưng trong khi thử vén màn “đạo giải bài”, nội dung đã mở rộng thành một thảo luận dài hơn. Tác giả cũng có phần bất ngờ vì bài viết trở nên dài như vậy, nhưng vẫn hy vọng nó có ích cho người đọc. Xin cảm ơn các thầy Wang Jinsheng, Ma Hongchun ở trường trung học đã thường xuyên động viên và quan tâm, cảm ơn huấn luyện viên Tang Wenbin và Hu Weidong của đội tuyển đã chú ý tới bài viết này, cũng như những người âm thầm ủng hộ tác giả. Mọi góp ý, phát hiện lỗi hoặc trao đổi thêm có thể gửi tới Zhang Kunwei tại aceeca.135531@gmail.com.

9 Nguồn bài tập

Ngoài các bài kinh điển, một số ví dụ trong bài lấy từ Andrew Stankevich’s Contest. Vì số lượng bài được nhắc tới khá nhiều, bản gốc chỉ giữ lại phần nội dung cốt lõi liên quan đến chủ đề của bài viết; người đọc quan tâm có thể tìm đề gốc, lời giải hoặc mã nguồn tương ứng.²

¹Cũng như khi không giải được một bài, ta thường không nên vội nghi ngờ rằng nó là một mệnh đề bất khả quyết nào đó.

²Khi trích dẫn nhiều bài, tác giả có dịch tên và bối cảnh đề. Để tiết kiệm thời gian đọc, bản gốc chỉ giữ phần liên quan trực tiếp đến chủ đề; những ai quan tâm tới bối cảnh chi tiết nên đọc đề gốc và tài liệu liên quan.